

SEVEN DESIGN PATTERNS

A Technical Reference for AI Engineering Interviews and Production Systems

Builds on Eugene Yan's framework for LLM-based products, consolidated around the boundary enforcement layer, and extended with the two patterns that dominate 2026-era interviews for agentic systems: Agentic Evals and LLM Observability.

- 01 **RAG** : Grounding generation in retrievable, verifiable context
- 02 **Fine-tuning** : Specialising weights when prompting hits a ceiling
- 03 **Caching** : Cutting latency and per-token cost with reuse
- 04 **Guardrails & Defensive UX** : Enforcing safety at the system boundary and the interface
- 05 **User Feedback** : Closing the loop into a self-improving data flywheel
- 06 **Agentic Evals** : Scoring trajectories, tool calls and multi-step plans
- 07 **LLM Observability** : Tracing, monitoring and alerting on live production traffic

Introduction

Building a demo takes an afternoon. Building a production LLM system that holds up under real traffic, real cost constraints and real failure modes takes a different discipline entirely. This guide is written for that discipline. It is organised around seven patterns that recur, in some combination, in nearly every AI engineering system design interview. Guardrails and Defensive UX are treated as a single chapter here because they are two layers of the same job: constraining what a model is allowed to do and gracefully handling what happens when it still gets something wrong. Agentic Evals and LLM Observability are included because they have become table stakes as interviews increasingly involve multi-step agentic systems rather than single prompt-response pairs.

Each chapter goes deeper than a definition. You will find the underlying mechanism, the concrete technical variants with their trade-offs, a dedicated section on what changes when the pattern moves from a notebook to a production service handling real traffic, and a representative interview question with an explicit note on what the interviewer is probing for. Measurement is not treated as a standalone chapter here; it is the connective tissue running through every pattern. Every chapter states explicitly how you would gate, monitor or verify that pattern in production, rather than deferring that discussion to a single evals section.

Two axes run through every pattern. Some patterns exist primarily to improve output quality: RAG, Fine-tuning, Agentic Evals. Others exist primarily to control cost and risk once quality is acceptable: Caching, Guardrails and Defensive UX, Observability. User Feedback sits at the centre and feeds both axes over time. Strong interview answers name this trade-off explicitly instead of treating every pattern as a free win.

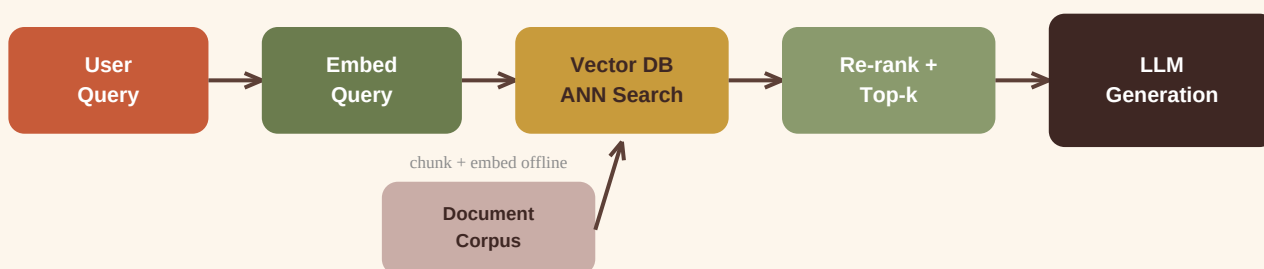
Pattern	Primary Goal	Axis
RAG	Ground output in external data	Quality
Fine-tuning	Specialise model behaviour	Quality
Caching	Reduce latency and cost	Cost / Risk
Guardrails & Defensive UX	Enforce safety, degrade gracefully	Cost / Risk
User Feedback	Build the data flywheel	Both
Agentic Evals	Score trajectories, not just answers	Quality
LLM Observability	See what happened in production	Cost / Risk

01

RAG: Retrieval-Augmented Generation

RAG fetches relevant data from outside the model's parameters and injects it into the context window, grounding generation in retrievable evidence and reducing hallucination. It is also far cheaper to keep a retrieval index current than to retrain a model, which is what makes RAG the default answer for incorporating recent or proprietary knowledge. This is, by a wide margin, the single most frequently posed case study in AI engineering interviews.

RAG Pipeline: Retrieve, Rank, Generate



1a. Naive Single-Pass RAG

Embed the query with the same model family used to embed the corpus, run approximate nearest neighbour search (HNSW or IVF-PQ indexes in stores such as pgvector, Pinecone, Weaviate or Qdrant), retrieve the top-k chunks by cosine similarity, and concatenate them into the prompt. Chunk size is the first real design decision: 256 to 512 tokens with 10 to 20 percent overlap is a reasonable default for prose, while structured documents (contracts, tables) often need semantic or layout-aware chunking rather than fixed-size windows. Naive RAG degrades on ambiguous queries, multi-hop questions that require synthesising several documents, and corpora with poor structural boundaries.

1b. Hybrid Retrieval RAG

Combine dense vector retrieval with sparse lexical search, typically BM25, and fuse the two ranked lists with reciprocal rank fusion or a learned cross-encoder re-ranker. This directly addresses a well-documented weakness of dense embeddings: they generalise away exact identifiers such as product SKUs, legal citation numbers, and rare proper nouns that a keyword index matches trivially. In practice a two-stage pipeline, cheap bi-encoder retrieval to fetch a wide candidate set of 50 to 100 chunks followed by a cross-encoder re-ranker to select the final top 5 to 10, consistently outperforms either method alone on standard retrieval benchmarks.

1c. Agentic and Iterative RAG

The model judges whether retrieved context is sufficient and, if not, reformulates the query and retrieves again, sometimes decomposing a complex question into sub-questions first. Self-RAG trains the model to emit special retrieval and critique tokens that trigger on-demand retrieval and self-assessment. Corrective RAG (CRAG) adds a lightweight relevance grader on retrieved documents and falls back to a web search when the internal knowledge base has insufficient coverage. Both add one to three extra round trips of latency and cost in exchange for materially better answers on multi-hop and ambiguous queries, so they should be reserved for query types where naive RAG measurably underperforms.

PRODUCTIONISING THIS PATTERN

- Chunking and re-indexing are recurring operational costs, not one-time setup. Design an incremental indexing pipeline (diff-based re-embedding of changed documents only) instead of a full re-embed on every content update, especially past a few hundred thousand documents.
- Track retrieval quality separately from generation quality using recall@k and mean reciprocal rank against a labelled query set. A RAG system can fail purely at retrieval while the LLM generation step looks fine in isolation, and conflating the two wastes debugging time.
- Embedding model versioning matters: if you change embedding models, old vectors and new query embeddings live in different geometric spaces and similarity scores become meaningless until you fully re-embed the corpus.
- Add source attribution and freshness metadata (document date, version) to every chunk so the generation step can reason about staleness and the UX layer can cite sources.
- At scale, ANN index build and query latency become first-class SLOs. HNSW gives faster query latency at higher memory cost, IVF-PQ trades recall for a smaller memory footprint. State this trade-off explicitly if asked about scaling past tens of millions of vectors.

INTERVIEW QUESTION

Design a system that lets employees search and ask questions over 50,000 internal company documents (policies, procedures, contracts). Documents are updated weekly. How would you architect this?

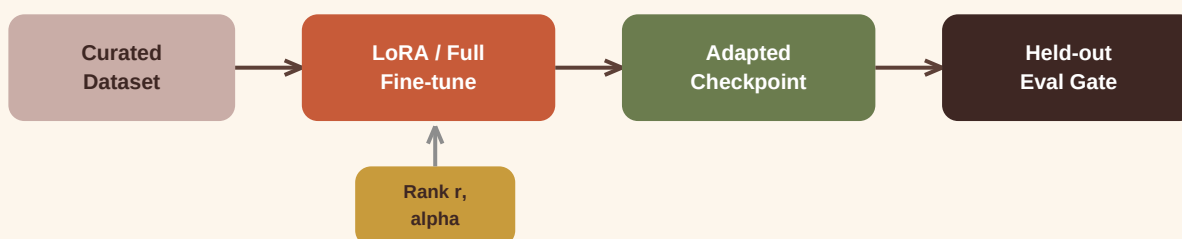
What they are testing: Chunking strategy, embedding model choice, hybrid retrieval combining BM25 and dense vectors, incremental index updates for the weekly refresh, handling multi-document questions, citation and attribution in responses, and access control since not every employee should see every document.

02

Fine-tuning: Specialising the Model

Fine-tuning adapts a pre-trained model's weights using task-specific demonstrations. Where prompting and RAG inject knowledge at inference time, fine-tuning bakes behaviour directly into the weights: consistent style, structured output adherence, domain-specific reasoning patterns, and shorter prompts since learned behaviour replaces lengthy instructions. The cost is curated data, compute, evaluation overhead and an additional model artifact to version and serve.

Parameter-Efficient Fine-tuning Pipeline



2a. Full Fine-tuning

Every parameter is updated on the task-specific dataset. This gives maximum expressiveness and is appropriate when the target domain diverges significantly from the base model's pretraining distribution, for example adapting a general-purpose model into a specialised biomedical or legal model. The costs are real: full-parameter gradients and optimiser states require several times the model's parameter count in GPU memory, catastrophic forgetting of general capabilities is a genuine risk without careful data mixing, and you now own a full model checkpoint to serve. In most production interviews, proposing full fine-tuning without justifying why PEFT is insufficient is treated as a red flag rather than a strong answer.

2b. Parameter-Efficient Fine-tuning (PEFT)

LoRA injects trainable low-rank decomposition matrices (rank r , typically 8 to 64) into the attention projection layers while freezing the base weights, updating well under 1 percent of total parameters. QLoRA adds 4-bit quantisation of the frozen base model, enabling fine-tuning of a 70B-parameter model on a single high-memory GPU. Because the base weights never move, you can serve many task-specific adapters from one shared base model and hot-swap them per request, which is the standard production pattern for multi-tenant fine-tuning. This is the default answer interviewers expect unless you can justify a reason for full fine-tuning.

2c. Instruction and Preference Tuning

RLHF trains a separate reward model on human preference pairs, then optimises the policy model against that reward using PPO. DPO (Direct Preference Optimisation) reformulates the same objective as a single supervised loss directly on preference pairs, removing the reward model and the RL loop entirely, which is why it has largely displaced RLHF for practical fine-tuning pipelines outside frontier labs. These methods tune for preferred behaviour: helpfulness, tone, refusal calibration, rather than narrow task accuracy, and are a distinct lever from task-specific SFT. Naming this distinction explicitly signals depth in an interview.

PRODUCTIONISING THIS PATTERN

- Always run a prompt-engineering and RAG baseline first and quantify the gap. Fine-tuning has the highest fixed cost of any pattern in this guide; skipping the baseline is the single most common mistake candidates describe.
- Data quality dominates data quantity for PEFT. A few hundred to low thousands of high-quality, diverse examples routinely outperform tens of thousands of noisy ones, and deduplication plus manual review of a sample is worth the time.
- Hold out an eval set that never touches training, and gate every fine-tuned checkpoint behind it before it reaches production, using the same held-out evaluation discipline you would apply to any model or prompt change.
- Plan for adapter lifecycle management: versioning, A/B testing new adapters against the incumbent on live traffic, and a fast rollback path if a fine-tune regresses on a task you did not explicitly test for.
- Catastrophic forgetting is real even with PEFT at high rank or long training runs. Include a slice of general-capability examples in the training mix and evaluate on both the target task and a general benchmark.

INTERVIEW QUESTION

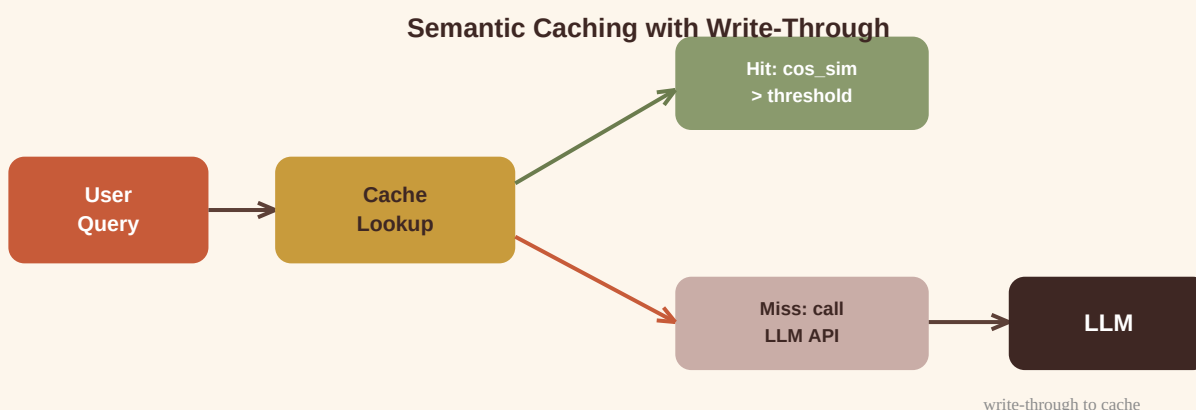
Your model produces correct answers but in a verbose, inconsistent format that downstream systems struggle to parse. Prompt engineering has helped but not enough. What would you do?

What they are testing: Do you reach for fine-tuning as a second tool, not a first? Can you articulate the LoRA versus full fine-tuning trade-off? Do you mention data curation for the desired output format, an eval gate for format adherence, and the risk of degrading content quality while fixing format?

03

Caching: Reducing Latency and Cost

LLM inference is expensive in both latency and dollars per token. Caching stores and reuses previously computed results so repeated or near-duplicate queries never reach the model. In high-traffic systems, a well-tuned caching layer routinely cuts inference spend by 30 to 80 percent and drops p50 latency from seconds to tens of milliseconds on cache hits. Interviewers read caching as evidence you think about unit economics, not only accuracy.



3a. Exact-Match Caching

Hash the normalised prompt (whitespace-trimmed, case-folded where safe) and use it as a key in Redis or Memcached. If the same key is seen again, return the cached response without ever calling the model. Hit rates are low for free-form conversational text where phrasing varies, but high for automated pipelines and batch jobs with canonical inputs. It is trivial to implement, carries zero risk of returning a semantically wrong answer, and should be the first layer in any caching stack because it is nearly free.

3b. Semantic Caching

Embed incoming queries and search the cache by cosine similarity rather than exact string match. If a new query's embedding falls within a similarity threshold of a cached query, typically 0.90 to 0.97 depending on tolerance for false positives, the cached response is returned. This lifts hit rates substantially for conversational traffic where users ask the same question in different words. The threshold is the entire design problem: set it too low and you serve a wrong cached answer to a different question, set it too high and hit rate collapses back toward exact-match levels. Tune it empirically against a labelled set of near-duplicate and non-duplicate query pairs, and treat it as a metric you monitor in production, not a value you set once at launch.

3c. Prefix and KV Caching

This operates at the model-serving layer, not the application layer. When multiple requests share a common prompt prefix, a long system prompt, few-shot exemplars, or a large retrieved context reused across turns, the transformer's key-value attention states for that prefix are computed once and reused, avoiding redundant forward passes. Most major inference providers expose this automatically or via an explicit cache-control parameter, and the savings on time-to-first-token can exceed 80 to 90 percent for long shared prefixes. Mentioning this distinguishes candidates who understand model-serving internals from those who only think in application-layer terms.

PRODUCTIONISING THIS PATTERN

- Cache invalidation is the hard problem, not the lookup. If the underlying knowledge base or RAG index changes, stale cache entries silently serve outdated answers. Tie cache TTL to your content update cadence, or invalidate keyed on a content version hash rather than time alone.
- Layer the three caching types rather than choosing one: exact-match first for near-zero cost, semantic cache second for paraphrase coverage, prefix caching underneath both for anything that still reaches the model.
- Monitor cache hit rate as a first-class production metric alongside cost per request. A sudden drop in hit rate is often the earliest signal of a shift in user query distribution.
- Be explicit that caching is unsafe for personalised or time-sensitive responses (account balances, live inventory) unless the cache key includes the relevant personalisation or timestamp dimension. This is a common trap interviewers set deliberately.

INTERVIEW QUESTION

Your LLM-powered FAQ system handles 100,000 queries per day, but 60 percent are variations of the same 500 questions. Inference costs are unsustainable. How would you redesign this?

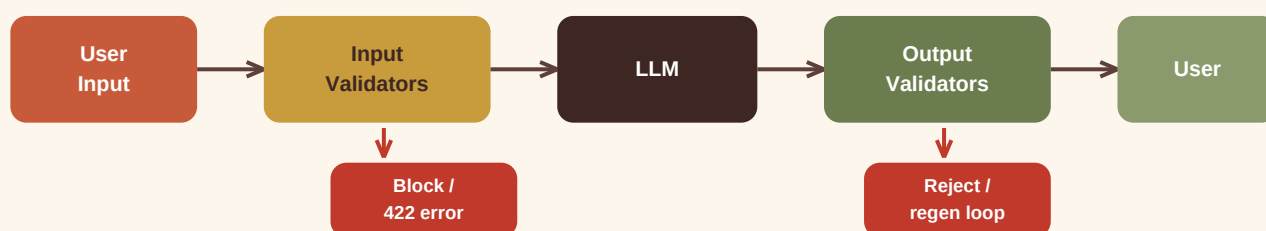
What they are testing: Do you layer exact-match and semantic caching? How do you handle cache invalidation when underlying knowledge changes? Do you mention TTL policy, cache warming, the risk of serving stale answers, and a rough cost and hit-rate calculation to justify the design?

04

Guardrails and Defensive UX: The Safety Boundary

Guardrails and Defensive UX are two layers of the same job and are treated together here because a strong interview answer never separates them. Guardrails are validators that constrain the inputs a model accepts and the outputs it produces, enforced automatically on every single request, blocking, rewriting or flagging problematic content before it reaches a user or a downstream system. Defensive UX is the acknowledgement that guardrails will not catch everything: it is a design strategy for the interface, built on the assumption that inaccuracies and hallucinations will still occasionally reach the user, and asks how the product should behave when that happens. Guardrails are the system's immune system; Defensive UX is what the patient feels while it is working. Interviewers frequently probe both in the same question because a system with strong guardrails and a brittle UI, or a gentle UI sitting on top of no runtime enforcement, both fail in production.

Guardrails as Runtime Enforcement



4a. Input Guardrails

These mitigate prompt injection, jailbreak attempts, toxic input and off-topic abuse before the prompt reaches the model. A layered implementation runs fast deterministic checks first, blocklists, regex patterns, length limits, then escalates to model-based classification for anything ambiguous: a moderation classifier such as OpenAI's Moderation endpoint or a dedicated prompt-injection detector, and a topic classifier that rejects out-of-scope queries before they waste a generation call. The core trade-off is false positive rate against false negative rate: over-aggressive filtering degrades legitimate use, under-aggressive filtering creates real safety exposure, and this threshold should be tuned against a labelled adversarial test set, not chosen arbitrarily.

4b. Structural and Syntactic Output Guardrails

These validate that output conforms to an expected schema, valid JSON, a syntactically correct SQL query, a response constrained to a fixed set of categories, before it is passed downstream. Constrained decoding libraries such as Outlines, Guidance and Instructor bias the token sampling distribution at generation time so the output is guaranteed to match a grammar or JSON schema, which is strictly more reliable than generating freely and validating afterward. When constrained decoding is unavailable, a generate-validate-retry loop with the validation error fed back into the next prompt is the standard fallback, typically capped at two or three retries before failing closed.

4c. Semantic and Factuality Guardrails

These confirm the output is semantically relevant to the input and factually grounded in provided context. For RAG systems specifically, this means faithfulness checking: does the generated response only assert claims that are entailed by the retrieved documents. Implementation approaches include NLI (natural language inference) models scoring entailment between the context and the response, embedding similarity between the answer and the source chunks, and dedicated hallucination-detection models. These are essential wherever factual errors carry real consequences, and are one of the highest-signal things you can add to a RAG answer in an interview.

4d. Graceful Degradation

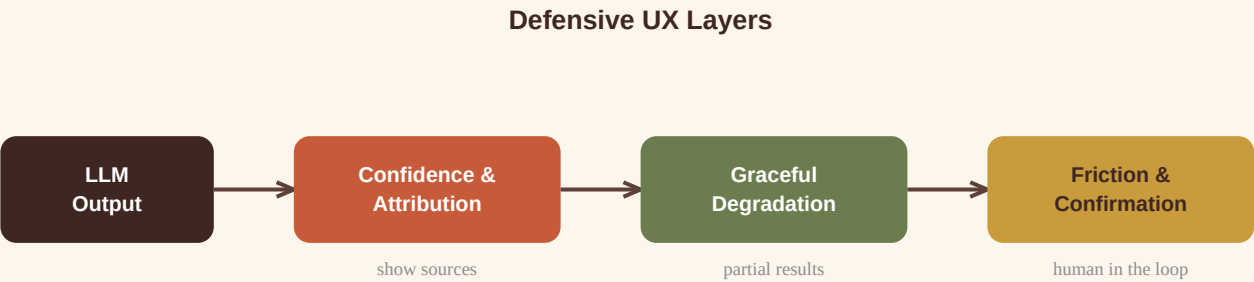
This is where Defensive UX picks up from guardrails. When the model is uncertain or retrieved context is thin, the system should degrade visibly rather than serve a confident hallucination that happened to pass every automated check: partial results with explicit caveats, surfaced confidence indicators where meaningful, and an offer to escalate to a human agent. This requires deliberately training or prompting the model to say it is unsure rather than fabricating a plausible-sounding answer, since models default to fluent completion regardless of actual certainty, and no guardrail can fully catch a confidently wrong answer that is syntactically and semantically well-formed.

4e. Attribution and Citation Surfacing

Showing the sources or reasoning that support a response lets users verify claims themselves rather than trusting blindly, which is a cheaper and more scalable defence than any single automated guardrail. In RAG systems this means linking directly to the retrieved chunks that informed the answer. In agentic systems it means exposing the tool calls and intermediate steps taken. Inline citation is now a standard user expectation, set by products such as Perplexity and modern chat assistants, and its absence is a frequent gap in candidate designs.

4f. Friction by Design

Deliberately inserting confirmation steps before high-stakes actions is the last line of defence when both guardrails and degradation signals fail to catch a problem, ensuring a hallucination cannot silently propagate into the real world. Any system that sends emails, executes code, makes purchases or modifies a database needs an explicit review-and-confirm step before the action fires. This extends to preview-before-publish workflows and presenting alternatives rather than a single take-it-or-leave-it output, keeping a human in the loop for consequential actions while allowing full automation for low-stakes ones.



PRODUCTIONISING THIS PATTERN

- Guardrails add latency on every request, not just the ones that fail. A cheap classifier adds single-digit milliseconds, an LLM-based semantic guardrail can add hundreds of milliseconds to a second, so reserve the expensive checks for high-stakes actions.
- Fail closed on high-stakes actions (financial transactions, destructive database operations) and fail open with a visible warning on low-stakes ones. Treating all failures identically is a common interview gap.
- Log every guardrail trigger with the input, the rule that fired, and the action taken. This log is your primary source for tuning false positive rate over time and for post-incident review.
- Defensive UX decisions should be driven by the cost-of-error matrix for the specific task, not applied uniformly. A wrong grocery-list suggestion and a wrong medication dosage warrant completely different levels of friction.
- Confidence indicators must be calibrated, not decorative. A model that claims 90 percent confidence should be right roughly 90 percent of the time on that claim category; uncalibrated confidence signals erode trust faster than no signal at all.
- Coordinate the two layers explicitly: a guardrail that silently rewrites output while the UX implies the raw model response is shown creates a trust gap the moment a user notices the mismatch, and instrumenting how often users edit or discard AI output feeds directly into Pattern 05, User Feedback.

INTERVIEW QUESTION

You are building an LLM-powered SQL query generator for a business analytics tool. Users describe what they want in natural language, and the system generates and executes SQL. What could go wrong, and how would you guard against it end to end, from the moment the query hits the model to the moment a result is shown to the user?

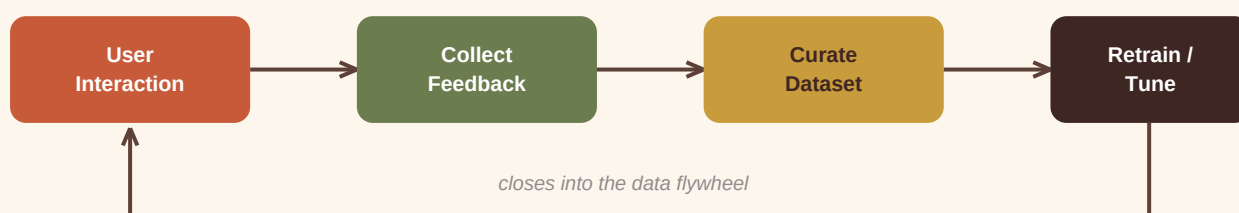
What they are testing: Input guardrails against injection attempts, structural guardrails validating SQL syntax before execution, semantic guardrails checking the SQL matches user intent, safety guardrails preventing destructive statements, least-privilege database access, and a defensive UX layer that shows the generated SQL before execution rather than silently running it.

05

User Feedback: Building the Data Flywheel

Collecting feedback closes the loop across every other pattern: feedback improves your eval set, better evals identify what to fine-tune, fine-tuning improves output, improved output generates more positive feedback. Without this loop a system ships at a fixed quality level and slowly degrades as the world around it changes. With it, the system compounds.

User Feedback Flywheel



5a. Explicit Feedback

Direct user signals: thumbs up or down, star ratings, free-text corrections. Clean and unambiguous, but participation is typically only 1 to 5 percent of interactions and skews toward users who are either very satisfied or very frustrated, a J-curve that biases naive aggregate scores. Maximise participation with low-friction, contextual prompts (one tap, asked immediately after the interaction) and specific questions rather than generic satisfaction surveys.

5b. Implicit Feedback

Behavioural signals inferred from actions: regeneration requests, copy or paste events, edit distance on a suggested draft, task completion rate, session abandonment. Volume is far higher than explicit feedback and captures users who never click a rating button, but the signal is noisier: a regeneration might mean the user was exploring alternatives, not that the first answer was bad. Calibrate implicit signals against a sample of explicit feedback before trusting them as a standalone quality metric.

5c. Correction Capture

The highest-value feedback type: logging the specific corrected version when a user fixes a bad output. An edited email draft, a corrected code snippet, a modified SQL query, each (original, corrected) pair is effectively free, in-distribution fine-tuning data. Building correction capture into the product itself, edit-in-place interfaces and explicit submit-corrected-version flows, is one of the highest-ROI investments in the flywheel. The constraint is privacy: corrections often contain sensitive content and require appropriate consent and anonymisation before they enter a training pipeline.

PRODUCTIONISING THIS PATTERN

- Guard against feedback loops that reinforce existing bias: if you only fine-tune on corrections from your most engaged users, the model increasingly optimises for that subgroup's preferences.
- Separate the feedback collection pipeline from the training pipeline with a manual or automated curation and deduplication step in between. Raw feedback should never flow directly into training without a quality filter.
- Address the cold-start problem explicitly: before you have enough production feedback, seed the eval set and initial fine-tune data from synthetic examples, red-teaming sessions, or a small paid annotation batch.
- Track feedback loop latency (time from user correction to that correction influencing the live model) as an explicit metric. A flywheel that takes six months per turn barely qualifies as a flywheel.

INTERVIEW QUESTION

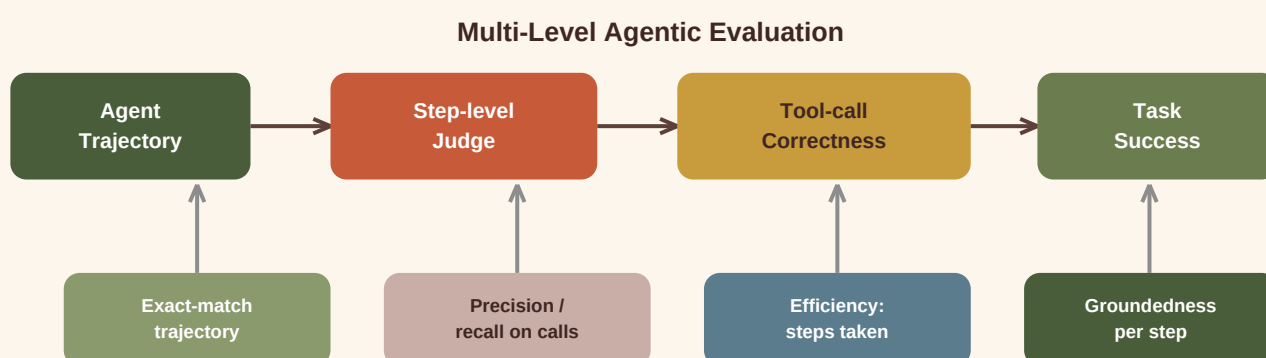
Your AI writing assistant has been live for six months. Engagement is steady but not growing, and you suspect output quality has plateaued. How would you design a feedback system to continuously improve the model?

What they are testing: Can you design a full flywheel: explicit and implicit signals, correction capture, a curation pipeline, and the eval framework that confirms each iteration actually improves things? Bonus points for naming cold-start problems, feedback bias, and privacy constraints unprompted.

06

Agentic Evals: Scoring the Trajectory, Not Just the Answer

Standard evals score a single input-output pair. Agentic systems make this insufficient: an agent that reaches the right final answer by calling the wrong tool, in the wrong order, five more times than necessary, has a serious production problem that final-answer accuracy alone will never surface. Agentic evals score the full trajectory: the sequence of reasoning steps, tool calls, tool results and intermediate decisions the agent took to arrive at its answer. This is one of the fastest-growing areas of interview questioning as agentic systems move from demos into production.



6a. Trajectory Evaluation

Scores the path taken, not only the destination. Exact-match trajectory evaluation checks whether the sequence of actions matches a labelled gold trajectory step for step, which is strict and often too rigid since multiple valid paths can reach the same correct answer. A more practical approach uses an LLM judge to assess whether each step was reasonable given the state at that point, independent of whether it matches a single reference path. This is the generalisation of LLM-as-judge scoring to a multi-step setting: instead of judging one output, the judge walks the trajectory and scores coherence, necessity and correctness of each step.

6b. Tool-Call Correctness

Precision and recall computed over the set of tool calls the agent made versus the set it should have made, given the task. Precision catches unnecessary or hallucinated tool calls, for example calling a search API when the answer was already in context. Recall catches missed steps, for example failing to call a verification tool before taking a destructive action. A further layer checks argument correctness: did the agent call the right tool with the right parameters, since a correct tool choice with malformed or wrong arguments is functionally a failure even though it may look correct at a glance.

6c. Efficiency and Groundedness Metrics

Efficiency metrics track the number of steps, tool calls and tokens consumed to reach a correct outcome, since two agents that both succeed are not equally good if one takes twelve steps and the other takes four. This directly maps to production cost and latency. Groundedness metrics check, at each step, whether the agent's stated reasoning is actually supported by the tool results it has received so far, catching a specific and dangerous failure

mode where an agent's chain-of-thought fabricates a justification disconnected from what its tools actually returned.

PRODUCTIONISING THIS PATTERN

- Build a labelled trajectory benchmark early, even a small one of 30 to 50 tasks with expected tool-call sequences. Without it you can only observe that an agent succeeded or failed, never why, which makes debugging production incidents extremely slow.
- Log the full trajectory (every intermediate reasoning step, tool call, tool result) for every production request, not just the final answer. This is the raw material both for agentic evals and for the observability pattern that follows.
- Set a hard step-count or token-budget ceiling per task and treat exceeding it as a failure mode in its own right, since an agent that loops indefinitely is a cost and reliability risk independent of whether it eventually produces a correct answer.
- Distinguish evaluation of the plan (did the agent choose a sound approach) from evaluation of execution (did it carry out that approach correctly), since a good plan poorly executed and a bad plan well executed require completely different fixes.

INTERVIEW QUESTION

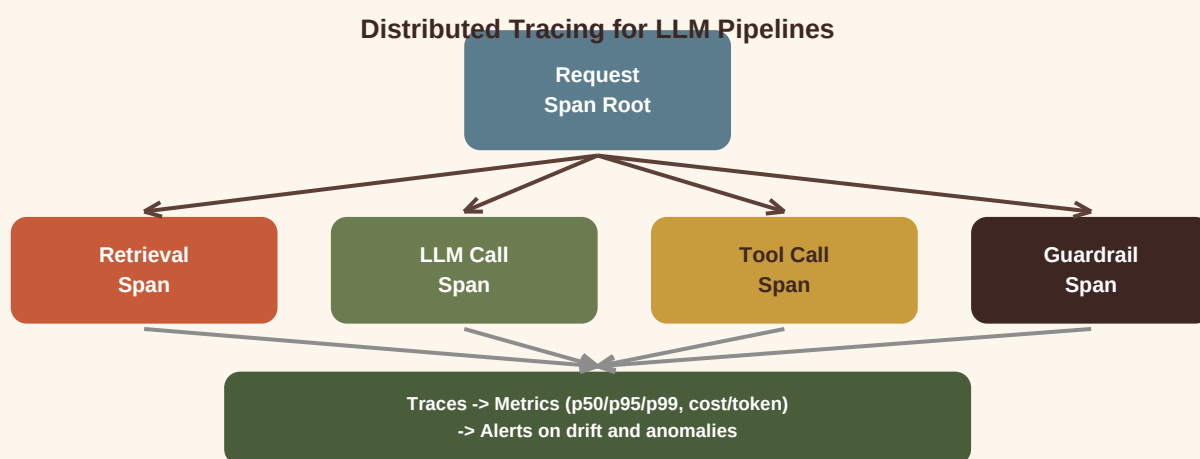
You have deployed an agent that books travel by searching flights, comparing prices and submitting a booking through an API. In testing, 85 percent of bookings are correct, but you suspect the failures are concentrated in a specific pattern. How would you diagnose this using evaluation rather than manual log review alone?

What they are testing: Do you propose trajectory-level evaluation rather than only checking final booking correctness? Do you separate tool-call precision and recall to isolate whether failures are search errors, comparison logic errors, or booking API misuse? Do you mention grounding checks to catch cases where the agent booked a flight its own search results did not actually support?

07

LLM Observability: Seeing What Happened in Production

Evals and agentic evals tell you what a system will likely do before you ship it. Observability tells you what it actually did, for a specific user, on a specific request, right now. The two are complementary and frequently confused in interviews: evals establish confidence before deployment, observability provides ground truth after it, and a mature answer keeps that distinction explicit.



7a. Tracing and Span-Level Logging

Every request is captured as a structured trace, following the OpenTelemetry model: a root span for the overall request, with child spans for each retrieval call, each LLM generation call, each tool invocation and each guardrail check, all tagged with inputs, outputs, latency and token counts. This lets you attribute a failure to the exact step that caused it rather than treating the pipeline as an opaque black box. Tools such as LangSmith, Langfuse, Arize Phoenix and standard OpenTelemetry-compatible backends implement this pattern for LLM pipelines specifically, adding LLM-aware concepts like prompt and completion capture on top of generic distributed tracing.

7b. Token, Cost and Latency Monitoring

Dashboards tracking token usage, dollar cost and latency percentiles (p50, p95, p99, since averages hide the tail behaviour that actually drives complaints) broken down per route, per model and per customer segment. This surfaces a specific, common production failure: a single prompt template or a single customer silently becoming disproportionately expensive, often because a chain grew an extra retry loop or a context window quietly ballooned. Rate limiting and budget alerts (RPM and TPM caps, per-tenant spend ceilings) belong in this layer, guarding against denial-of-wallet risk where a bug or an abusive user can generate an unbounded bill.

7c. Drift and Anomaly Detection

Monitoring output distributions, retrieval quality and user feedback signals over time to catch silent degradation that no single request would reveal. Examples: embedding model drift where an upstream provider silently updates a model and the vector space shifts under you, a knowledge base going stale as source documents age past a freshness threshold, or a gradual shift in the distribution of user queries that pushes traffic outside your evaluated distribution. Statistical approaches range from simple threshold alerts on rolling eval scores to

population-level distribution comparisons (KL divergence or population stability index) between current and baseline traffic.

PRODUCTIONISING THIS PATTERN

- Observability and agentic evals should share the same trace schema. If your evaluation harness and your production tracing log different fields, you cannot replay a production trajectory directly into your eval pipeline when debugging an incident, which is the single most common integration gap in real systems.
- Redact or tokenise sensitive content (PII, credentials, proprietary data) at the tracing layer before it is persisted, not after. Full prompt and completion logging is enormously useful for debugging but creates a real compliance surface if handled carelessly.
- Alert on leading indicators, not just lagging ones. A rising retry rate or a falling cache hit rate often predicts a cost or quality incident hours before user-facing complaints arrive.
- Sampling strategy matters at scale: full tracing on every request is expensive at high volume, so a common pattern is full tracing on a statistically significant sample plus lightweight metrics-only logging on the remainder, with automatic upgrade to full tracing when an anomaly is detected.
- Set SLOs, not just dashboards: an explicit p95 latency target and an explicit cost-per-request ceiling that page an on-call engineer when breached, exactly as you would for any other backend service.

INTERVIEW QUESTION

Your agentic system has been in production for three months. Yesterday, a customer reported the agent gave a wrong answer, but you have no way to reproduce it and the logs only show the final response. How would you redesign your observability so this does not happen again, and how would you diagnose the reported incident with what little you have?

What they are testing: Do you propose full-trace logging (root span plus child spans for retrieval, generation, tool calls) rather than final-output-only logging? Do you connect this to replayability, being able to feed a captured trace back into an eval harness? Do you mention cost and privacy trade-offs of full tracing at scale, and distinguish this from the separate problem of preventing the failure via evals versus merely detecting it via observability?

Quick Reference Cheat Sheet

A last pass before an interview. For each pattern: its core technical variants, and the single thing the interviewer is actually testing when they ask about it.

Pattern	Variants	Interviewer Is Testing
RAG	Naive, Hybrid (BM25 + dense), Agentic/iterative	Can you design an end-to-end retrieval pipeline?
Fine-tuning	Full, LoRA/QLoRA, RLHF/DPO	Do you know when not to fine-tune?
Caching	Exact-match, Semantic, KV/Prefix	Do you think about cost and invalidation at scale?
Guardrails & Defensive UX	Input/structural/semantic guardrails, Degradation, Friction	Can you enforce safety and handle failure gracefully?
User Feedback	Explicit, Implicit, Correction capture	Can you build a system that improves itself?
Agentic Evals	Trajectory, Tool-call precision/recall, Efficiency	Do you evaluate the path, not just the answer?
LLM Observability	Tracing, Cost/latency monitoring, Drift detection	Can you diagnose a live incident you cannot reproduce?

A Final Note on Interview Strategy

These seven patterns are not mutually exclusive; a strong case study answer typically combines four or five of them. A production RAG system needs guardrails to validate output, caching to control cost, defensive UX to handle failure at the interface, observability to see what happened after launch, and a feedback loop to keep improving, all measured against an eval harness you define as part of describing the pattern itself rather than bolting on afterward. If the system is agentic, agentic evals extend that measurement discipline to score the full trajectory, not just the final answer. The interviewer is testing whether you can justify why a specific combination fits the stated constraints, not whether you can recite the taxonomy. Open with how you would measure success, work through the patterns that address the specific failure modes of the problem, and close with how the system observes itself and improves over time.

Further Reading

- Eugene Yan, Patterns for Building LLM-based Systems and Products (eugeneyan.com)
- Anthropic, Building Effective Agents (anthropic.com)
- Chip Huyen, Building a Generative AI Platform
- OpenTelemetry documentation for distributed tracing fundamentals (opentelemetry.io)
- Google, People + AI Guidebook (pair.withgoogle.com)